# 松鼠备份 StarUML 手工绘图指导

本指导供你在 StarUML 中手工完成课程建模，内容依据松鼠备份 0.1.1 的需求与公开接口。先冻结当前接口和行为，再开始绘图；所有用例编号与01需求说明书一致。本文不包含生成的 UML 图、PlantUML、Mermaid 或可直接导入的模型。本轮仅更新绘图指导，模型和图仍待完成；这不表示课程禁止生成可编辑模型。推荐先完成一张用例图、一张核心类图、一张构件图、备份与恢复各一张顺序图和一张任务状态图，再插入实验报告对应栏目。

## 1 建立项目与通用操作

安装适用于 Mac 的 StarUML，并按其许可条件使用。新建项目后保存为 SquirrelBackup.mdj。右侧 Model Explorer 保存模型元素，左侧 Toolbox 提供绘图元素，Property Editor 设置名字和属性。先选模型或包，再通过 Model > Add Diagram > 对应图类型创建图；也可使用模型节点右键菜单。该流程依据官方 Managing Diagrams 文档。

建议用包分组：Requirements、Architecture、Core、Scenarios、States。图名按 UC_总体用例、CD_核心类、CMP_系统构件、SEQ_创建备份、SEQ_安全恢复、STM_自动任务命名。双击模型元素可编辑名字，类图可在 QuickEdit 添加属性和操作。不要把快捷键作为唯一操作方式，Mac 版本的键位可能不同，以菜单和右键可见项为准。

所有图统一使用清晰的中英文名字，避免把代码所有辅助函数塞进一张图。图片导出前检查文字和箭头不重叠；保留 mdj 源文件。若导出菜单与版本不同，在 File 中查找 Export 相关项或使用应用内帮助确认。不要用屏幕上看不清的缩略图替代建模文件。

## 2 用例图 UC_总体用例

在 Requirements 包中建立 Use Case Diagram。用 Toolbox 的 Actor 放置“本机用户”，使用 Use Case Subject 建立名为“松鼠备份”的系统边界，在边界内放置下表用例。参与者放在系统边界之外。这些元素的创建方法可参见官方 Use Case Diagram 文档。

| 编号 | 用例名 | 用户关联 | 需求 |
| --- | --- | --- | --- |
| UC-01 | 创建备份 | 是 | FR-01、03、04、12、13、14 |
| UC-02 | 恢复备份 | 是 | FR-02、03、04 |
| UC-03 | 验证完整性 | 是 | FR-15 |
| UC-04 | 查看内容 | 是 | FR-16 |
| UC-05 | 配置筛选 | 是 | FR-05 至 11 |
| UC-06 | 启动定时任务 | 是 | FR-18、19 |
| UC-07 | 启动实时任务 | 是 | FR-20、19 |
| UC-08 | 停止自动任务 | 是 | FR-21 |

用 Association 把用户连接到以上可操作用例。恢复备份和查看内容都必须验证完整性，分别添加 Include，方向从基础用例指向被包含的“验证完整性”。定时和实时任务调用创建备份，可各用 Include 指向“创建备份”，旁

边添加 Note 注明重复执行和停止条件。配置筛选是创建前的可选设置，不需要为显示复杂度硬加 extend；可关联用户并以 Note 说明影响创建与自动任务。

压缩、加密不必单独作为顶层用户目标，可在“创建备份”备注中列为可选处理。不要添加“云用户”“登录”“网盘”等未实现功能。系统时钟是实现机制，初版没有独立外部定时服务，因此不必新增虚构参与者。

检查：系统边界名字正确；用户位于外部；include 箭头指向验证用例；用例名是用户目标而非 C++ 函数；需求文档中的前置、基本、异常和后置条件能逐一对上。完成后插入需求说明书系统用例位置。

## 3 类图 CD_核心类

在 Core 包中建立 Class Diagram，使用 Class 元素放入 BackupService、FileScanner、ArchiveWriter、ArchiveReader、HuffmanCodec、CryptoEnvelope、Scheduler、BackupOptions、ScheduleOptions、Filter、Entry、Summary。FileType 可用 Enumeration。它们对应公开头文件，内部 Writer/Reader/Fd/TempDir 如需展示可另建辅助类图，避免主图过密。

双击类后通过 Add Attribute 和 Add Operation 填写属性和方法；可以使用右侧 Property Editor 核对 visibility、type 和 multiplicity。属性格式示例为 +compress: bool = true，操作示例为 +backup(source: path, archive: path, options: BackupOptions): Summary。这是 UML 展示签名，可适度省略 const、引用和默认回调，但实际方法名不能随意改动。

| 类 | 应填属性 | 应填操作 |
| --- | --- | --- |
| BackupService | 无持久属性 | backup、inspect、restore |
| FileScanner | 无持久属性 | scan(root, filter, log): vector<Entry> |
| ArchiveWriter | 无持久属性 | write(source, entries, output, compress, log) |
| ArchiveReader | 无持久属性 | read(plain, destination, log, collect): Summary |
| HuffmanCodec | 无持久属性 | encode(input): Bytes；decode(input, expected): Bytes |
| CryptoEnvelope | 无持久属性 | seal(plain, output, password)；open(archive, plain, password) |
| Scheduler | 无持久属性 | run(source, repository, backup, options, stop, log) |
| BackupOptions | filter、compress、password | 无 |
| ScheduleOptions | interval_seconds、keep、poll_seconds、watch、max_runs | 无 |
| Filter | excludes、path_pattern、name_pattern、types、min_size、max_size、since、until、owner | excluded、accepts |
| Entry | path、link、type、mode、uid、gid、mtime、mtime_ns、atime、atime_ns、size、xattrs；扫描专用device/inode/ctime/ctime_ns | 无 |
| Summary | entries、files、bytes、archive_bytes、items | 无 |

关系按下表连接。静态调用使用 Dependency，箭头指向被依赖者；不要用实体 Association 冒充长期拥有关系。组合关系实心菱形位于拥有者一侧。

| 起点 | 终点 | 关系 | 基数或备注 |
| --- | --- | --- | --- |
| BackupService | FileScanner | Dependency | 调用 scan |
| BackupService | ArchiveWriter | Dependency | 调用 write |
| BackupService | ArchiveReader | Dependency | 调用 read |
| BackupService | CryptoEnvelope | Dependency | 调用 seal/open |
| ArchiveWriter | HuffmanCodec | Dependency | 编码 |
| ArchiveReader | HuffmanCodec | Dependency | 解码 |
| Scheduler | BackupService | Dependency | 循环创建快照 |
| BackupOptions | Filter | Composition | 1 对 1 |
| Summary | Entry | Composition | 1 对 0..* |
| Entry | FileType | Dependency | type 枚举属性 |

将 Filter、Entry、Summary 归到值对象一排，业务服务居中，算法和格式类分列。静态工具类的方法标记为 static，BackupService 的业务方法不是 static。没有接口继承层次，不画 Generalization。如果组合已经显示成连线，可在类框中省略重复的对应属性以降低冗余，并在注释中说明。

检查：名字和源码一致；静态标记正确；拥有关系有基数；不要把两个入口各画成一套 BackupService 业务实现。Entry 的内存扫描字段与归档持久化字段需区分；以冻结后的 backup.h 为准，未写入归档的检测字段不得画成格式已经新增字段。完成后插入系统设计文档 3.1 系统对象模型。

## 4 构件图 CMP_系统构件

在 Architecture 包中建立 Component Diagram。摆放 GUI Executable、CLI Executable、Core Static Library 三个项目构件，外部构件为 Cocoa/AppKit、CommonCrypto/Security、macOS File System。GUI 与 CLI 都依赖 Core Static Library；GUI 额外依赖 Cocoa；核心库依赖密码库与文件系统。

在 Core 的 Note 中列出 Scanner、Archive、Huffman、Envelope、Scheduler 模块即可，不要把类图完全复制过来。接口可标注“C++ backup.h”，因为当前是静态库和 C++ 编译期链接；不要标注成已经实现的 REST、DLL 或稳定 C ABI。GUI 的后台队列与 C++ 方法调用发生在同一进程，不能画成远端服务。

如需子系统构件图，单独在 Core 包画“扫描与筛选”“树记录编解码”“密码封装”“任务调度”四个构件；依赖方向由调用方指向提供方。完成后插入总体设计 2.1；顶层图为必选，子系统图按教师要求补充。

## 5 顺序图 SEQ_创建备份

在 Scenarios 包建立 Sequence Diagram。使用 Lifeline 放置以下生命线，自左向右为 user、ui:SquirrelController、service:BackupService、scanner:FileScanner、writer:ArchiveWriter、codec:HuffmanCodec、envelope:CryptoEnvelope、filesystem。工具类生命线代表调用目标，不意味着源码创建了持久实例。

使用 Message 表达方法调用，返回使用 reply；按时间从上到下布局。官方 Sequence Diagram 文档说明可通过 messageSort 设置同步、异步和返回。项目中的 GUI 到工作队列属于异步调度；工作队列内部的核心方法调用是同步。

| 顺序 | 发送方到接收方 | 消息 |
| --- | --- | --- |
| 1 | user 到 ui | 点击开始备份，提交路径和选项 |
| 2 | ui 到 ui | 收集输入，禁用业务按钮，启动后台工作 |
| 3 | ui 到 service | backup(source, archive, options, log) |
| 4 | service 到 filesystem | 检查输出不存在且不在源树内 |
| 5 | service 到 scanner | scan(root, filter) |
| 6 | scanner 到 service | 返回条目列表 |
| 7 | service 到 writer | write(source, entries, tempTree, compress) |
| 8 | writer 到 filesystem | 安全打开文件、读一个块、检查变化 |
| 9 | writer 到 codec | encode(block)，仅开启压缩时 |
| 10 | writer 到 filesystem | 写块和树记录，完成后关闭 |
| 11 | service 到 envelope | seal(tempTree, sealed, password) |
| 12 | envelope 到 filesystem | 写完整性封装或密文及认证标签 |
| 13 | service 到 filesystem | fsync 和 link 独占发布 |
| 14 | service 到 ui | 返回 Summary 或异常 |
| 15 | ui 到 ui | 主队列显示结果、启用按钮 |

用 loop 片段包住文件和块处理；用 opt 标记压缩和加密可选分支；用 alt 表达成功与读取 / 权限 / 磁盘失败。失败分支返回错误并清理暂存，不发布新备份。不要把备份“成功提示”放在 link 成功之前。

# 6 顺序图 SEQ_安全恢复

生命线为 user、ui、service:BackupService、envelope:CryptoEnvelope、reader:ArchiveReader、codec:HuffmanCodec、filesystem。主要消息如下：

• 用户提交备份、密码和新目标路径。

• service 检查目标路径不存在，并创建私有暂存目录。

• envelope.open 读取外层头并计算全部 HMAC / 摘要。

• alt 校验失败：抛异常，不创建可见恢复目标；成功：继续第二遍解密 / 复制。

• reader.read 验证根记录、条目数量与相对路径。

• loop 各条目：验证父节点和类型，创建目录、文件、软硬链接或 FIFO。

• loop 普通文件各块：必要时 codec.decode，写入文件。

• reader 恢复非目录元数据，目录延迟逆序处理；检查 footer 和 EOF。

• service 使用 renamex_np(RENAME_EXCL) 发布恢复目录。

• 返回 Summary，UI 主队列显示完成。

在失败 alt 分支注明“原有目标不覆盖，私有目录尝试清理”。owner/xattr 恢复失败的警告属于可降级分支，不等于文件内容恢复失败。顺序图应表现先验证后解密，不要根据 CBC 机制臆造先试密码解密再认证。

# 7 状态图 STM_自动任务

在 States 包建立 Statechart Diagram。放置 Initial、Idle、Scanning、BackingUp、Waiting、RetryWaiting、Stopping 和 Final。对转换写“事件 [守卫] / 动作”：

| 起点 | 终点 | 触发与动作 |
| --- | --- | --- |
| Initial | Idle | 应用 / CLI 初始化 |
| Idle | Scanning | 启动实时任务 / 读取选项 |
| Idle | BackingUp | 启动定时任务 / 立即快照 |
| Scanning | BackingUp | 首次或稳定变化 |
| Scanning | Waiting | 无变化或变化未稳定 |
| BackingUp | Waiting | 成功 / 登记快照并按 keep 淘汰 |
| BackingUp | RetryWaiting | 失败 / 记录错误，保留旧快照 |
| Waiting | Scanning | 实时轮询到期 |
| Waiting | BackingUp | 定时间隔到期 |
| RetryWaiting | Scanning 或 BackingUp | 到期重试，依任务模式选择 |
| Waiting 或 Scanning | Stopping | stop 标志置位 |
| BackingUp | Stopping | 当前快照完成且 stop 已请求 |
| Stopping | Final | 输出统计并退出循环 |

状态名是对 Scheduler 控制流的建模抽象，代码没有单独的状态枚举。GUI还应以注释说明0.1.1的延迟关闭/退出请求：忙碌时记录，操作结束后执行；这不是Scheduler增加了独立后台守护状态。停止不会立即中断块写入，不能画成点击停止便立刻销毁快照。max_runs 达成也进入 Final。完成后插入系统设计动态建模章节。

# 8 插入报告与最终核对

在绘图前记录0.1.1源代码版本/提交号，复核公开接口、UC-01至UC-08及停止语义；涉及接口的修订完成后再冻结模型。将各图放到模板对应位置，保留图题和文字说明。需求文档使用用例图，设计文档使用构件、类、顺序和状态图。字体统一；连线避免穿过类框；一个页面过密则拆图。最终把 mdj 源文件和导出图片一起归档，报告中说明实际使用的 StarUML 版本和绘图人。

逐项检查：没有未实现网络功能；static 与非 static 区分正确；依赖箭头方向正确；组合关系有基数；方法名能在源码定位；异常和停止路径均有说明；图和报告版本号一致。没有完成绘图时不把“StarUML 模型已完成”填写到实验结论。

# 9 官方操作参考

以下仅用于确认 StarUML 的操作入口，项目元素、方法和流程来自本项目源码。菜单会随版本变化，以实际安装版本为准。

- Managing Diagrams：https://docs.staruml.io/user-guide/managing-diagrams
- Use Case Diagram：https://docs.staruml.io/working-with-uml-diagrams/use-case-diagram
- Class Diagram：https://docs.staruml.io/working-with-uml-diagrams/class-diagram
- Sequence Diagram：https://docs.staruml.io/working-with-uml-diagrams/sequence-diagram
- Statechart Diagram：https://docs.staruml.io/working-with-uml-diagrams/statechart-diagram